

MAE014 - Tóp. Ciência Dados B [Turmas MAP e EMA]

Preenchimento de Região em Matriz de Caracteres

Nome completo	DRE
Emanuel Cardoso	122033151
Alanna Figueiredo Simões	120053919
Felipe Rodrigues Schoeffer	123311188
Artur de Melo Brito	117224234
Matheus de Sousa	123536584

Data: 11/09/2026

Análise Crítica — Código Gerado com IA

1. O que a IA ajudou a produzir

- uma primeira versão funcional dos dois algoritmos pedidos (recursivo e com pilha de posições);
- a identificação inicial dos vizinhos por 4-conectividade e da condição de parada (limites da matriz e célula já preenchida);
- a arquitetura modular final (algoritmo → lista de eventos → reprodutor de texto / interface gráfica), depois de explicitado como requisito;
- os testes de estresse que comparam as versões ingênua e final.

2. O que precisou de revisão humana

O código bruto (reproduzido em `POST_MORTEM.md`) não deveria ser aceito sem revisão. Foi necessário:

- perceber que a matriz lida como lista de strings não pode ser alterada célula a célula em tempo constante (strings são imutáveis em Python);
- perceber que a recursão "pura" tem um limite de profundidade e que o enunciado pede regiões que podem ser grandes o bastante para estourá-lo — isso só ficou evidente ao montar um teste de estresse com uma sala vazia grande, não ao ler o código;
- questionar a capacidade "estimada" da pilha ingênua, que funciona nos exemplos pequenos do enunciado mas não é uma garantia matemática;
- verificar, célula a célula, o exemplo de figura fechada do enunciado (ver seção 3) antes de confiar cegamente no resultado do algoritmo.

3. Um problema encontrado no próprio enunciado (não no código da IA)

O enunciado traz duas matrizes em sequência — a primeira com o marcador X, a segunda aparentemente como o resultado esperado do preenchimento. Ao rodar o algoritmo final sobre a primeira matriz, o resultado bate com a segunda em todas as células, exceto uma: a célula da linha 2, coluna 3 (0-indexado), que no enunciado aparece preenchida, mas que — nas duas matrizes fornecidas — é um 1 cercado por 0 nos quatro lados (cima, baixo, esquerda, direita), ou seja, uma "ilha" de uma célula sem nenhuma ligação ortogonal com a região onde está o X.

Testamos a hipótese de que o enunciado usasse 8-conectividade (incluindo diagonais): nesse caso o preenchimento vaza pelas diagonais da própria

figura e toma a matriz inteira, o que não corresponde ao resultado mostrado no enunciado. Isso reforça que a figura foi pensada para 4-conectividade, e que a célula isolada no material de referência é, com grande probabilidade, uma inconsistência pontual do exemplo (bem plausível em uma arte ASCII copiada manualmente entre slides), não um sinal de que o algoritmo deveria ser outro. Optou-se por manter 4-conectividade e documentar esse achado, em vez de forçar o código a reproduzir uma inconsistência do material de apoio. Essa checagem virou o teste `test_figura_pequena_nao_extrapol_a_ilha_isolada` em `test_preenchimento.py`.

4. "Produza uma solução também" para o caso de labirinto/robótica

O enunciado cita o uso do preenchimento por inundação para navegação em labirintos (robótica) e pede uma solução também para esse caso. A maior matriz de exemplo do próprio enunciado já é, na prática, um labirinto: um emaranhado de corredores com o X no meio. Interpretamos o pedido como "demonstre que o mesmo algoritmo funciona nesse cenário", não como "escreva um algoritmo de busca de caminho diferente" — e usamos essa matriz (`matrizes/labirinto.txt`) como o terceiro exemplo testado, confirmando que o preenchimento por 4-conectividade também resolve corretamente esse caso (502 de 2.100 células preenchidas, sem vazar para fora dos corredores). O algoritmo de preenchimento por inundação é, de fato, a base do clássico "flood fill" usado em micromouse (robôs solucionadores de labirinto) para calcular distâncias até o objetivo — o que valida essa leitura do enunciado.

5. Complexidade e limitações conhecidas

- As duas implementações finais são $O(n)$ em tempo e espaço, n sendo o número de células da matriz — ver `CODE_REVIEW.md`.
- A versão recursiva depende de rodar em uma thread com pilha nativa maior e limite de recursão elevado. Isso tem um custo (criar uma thread por chamada) e uma limitação de concorrência documentada no `CODE_REVIEW.md` (as configurações alteradas são globais ao processo, não por thread). Para o uso previsto — um preenchimento por vez, disparado pela CLI ou pela interface gráfica — essa limitação não se manifesta.
- A interface gráfica (`interface_grafica.py`) depende do módulo padrão `tkinter`, que nem toda instalação mínima de Python inclui por padrão. Isso não foi possível validar de ponta a ponta neste ambiente de preparação (container sem `tkinter` nem servidor gráfico); a lógica foi revisada por leitura e é a mesma já validada no reprodutor de texto

(mesma lista de eventos, mesma matriz original), mas o grupo deve testar a janela em uma máquina com ambiente gráfico antes da entrega. Para não perder toda a cobertura de teste dessa parte, a validação da velocidade de reprodução (entrada de texto, sem nenhuma dependência de tkinter) foi extraída para `entrada_usuario.py` e é testada normalmente em `test_entrada_usuario.py`.

6. Conclusão crítica

O uso de IA acelerou a primeira versão dos dois algoritmos, mas essa primeira versão tinha exatamente os problemas que o enunciado pede para procurar: complexidade de escrita evitável, uma pilha dimensionada por estimativa em vez de por prova, e nenhuma consideração sobre o limite de recursão do Python — este último, na prática, o problema mais sério, pois não é uma questão de desempenho, e sim de a rotina recursiva simplesmente não terminar para entradas do tamanho que o próprio enunciado sugere. A revisão humana foi necessária tanto para corrigir esses pontos quanto para validar o algoritmo contra os exemplos fornecidos — inclusive para perceber uma pequena inconsistência no material de apoio, que só apareceu ao comparar célula a célula o resultado do código com o exemplo do enunciado.